

IFT320

SYSTÈMES D'EXPLOITATION

Travail pratique 4

Travail en équipe:

Pour les groupes de **deux** personnes exactement, les parties à réaliser sont :

1-a) b) c)

2-a) b) **c) d)**

Pour les personnes voulant être **seules**, les parties à réaliser sont :

1-a) b) c)

2-a) b)

Pour les personnes voulant être **en groupe de trois**, toutes les parties sont à faire. L'omission des parties supplémentaires vous pénalise de 20% de la note.

Les parties supplémentaires peuvent être faites en bonus par les personnes seules ou les équipes de 2. Les bonus sont plus payants pour les personnes seules (note pondérée sur un total plus bas).

Mode de remise: électronique, turnin web

Les trois derniers travaux pratiques du cours IFT320 portent tous sur NachOS (Not Another Completely Heuristic OS), un système d'exploitation minimaliste à but pédagogique. Au cours de ces TP, vous aurez à modifier différentes parties du système afin de leur rajouter des fonctionnalités.

À travers les années, une certaine quantité d'explications et de documentation a été rassemblée à propos de NachOS. Lisez le document d'introduction à NachOS pour l'ancien cours IFT518 (ancien_nachos_ift518.pdf).

Cependant, notez bien que dans ce document les anciens énoncés des travaux et les dates de remise indiqués ne sont plus valides. Les dates de remise pour cette session sont celles indiquées directement sur turnin web, et les énoncés actuels des travaux se trouvent ci-dessous.

Le seul système sur lequel fonctionne NachOS au département la machine virtuelle qu'on vous a fournie, vos travaux seront tous vérifiés sur cette plateforme, dans un script de tests. Ces tests

seront faits dans l'ordre dans lequel les questions sont posées; il est donc fondamental que vous complétiez chaque question avant de commencer la suivante. Pour chaque travail pratique, vous devrez soumettre par turnin sur tarin vos sources modifiées de NachOS. La structure de répertoires du dossier « nachos » qui vous est fourni doit demeurer intacte. Vous devez remettre le répertoire « nachos » au complet avec ses sous-répertoires.

NE CHANGEZ AUCUN NOM DE RÉPERTOIRE.

Toute demande de resoumission ou de recorection se verra octroyer une pénalité de 10% de la note.

Finalement, dans tous vos travaux, souvenez-vous de ne jamais modifier le répertoire **machine** et son contenu !

Nous copierons notre version à la place de la vôtre lors du train de test pour s'assurer de cela.

Travail Pratique 4 : Mémoire virtuelle

À propos du code de base...

Le code actuel est une solution imparfaite au devoir 3. En effet, elle ne respecte pas toutes les bonnes pratiques de code, et certains des numéros du devoir 3 ne sont pas implantés.

Ce code peut exécuter la grande majorité des tests du devoir 3, qui vous sont recopiés et fournis dans /tp4test, avec les tests supplémentaires pour le devoir 4. À noter, les algorithmes de planification ne fonctionnent pas parfaitement dans cette version, et l'algorithme par défaut est le tourniquet (Round Robin). Tous les tests du devoir #4 seront faits avec cet algorithme, pour obtenir des résultats intéressants.

L'implantation des appels système est faite directement dans le switch-case à deux niveaux qui se trouve dans exception.cc. Elle est simplifiée, notamment par la factorisation de quelques morceaux de code commun et par l'implantation de CopyFromUser et CopyToUser. Ces deux fonctions se trouvent dans le fichier copy.cc.

Une capacité supplémentaire a été ajoutée à l'appel système Exec, celle de passer un paramètre (un int) à un processus enfant lors de son exécution. Ceci sera utile pour le test exhaustif (**terat**) qui devrait pouvoir fonctionner à la fin de votre devoir.

Vérifications avant de commencer

Naviguez vers le répertoire **nachos/code/vm** . Ce répertoire est celui dans lequel vous allez travailler. Nachos devrait être compilé dans ce répertoire et tous vos tests devraient être effectués à partir de ce répertoire.

Compilez nachos à l'aide de la commande **make**.

N.B. : Il est probable que nachos ne compilera pas dans les autres répertoires. Compilez-le uniquement dans /vm.

Plusieurs scripts sont disponibles pour tester l'avancement de votre travail. Certains d'entre eux fonctionnent déjà et doivent fonctionner jusqu'à la fin de votre travail, d'autres commenceront à fonctionner avec l'avancement de votre travail. En voici une liste exhaustive :

Tests fonctionnels au départ :

tcreate, terror, texec, texit, tfull, tjoin, tlog, tminijoin, tread, treadwrite, trobin, tsqrt, twrite, tyield.

N.B. tjoin peut échouer, à cause d'un bug dans l'implémentation de la table de jointure. Ce bug ne devrait pas vous affecter avant l'implantation du numéro bonus pour le test exhaustif final (terat).

Ces tests sont identiques à ceux du devoir 3 et testent les mêmes aspects de l'implantation. Si vous avez des doutes quant au bon fonctionnement d'une partie spécifique du code après certaines modifications, ces tests peuvent aider faire des vérifications.

tsqrt est un nouveau test qui calcule l'entier précédent ou égal à la racine carrée d'un autre entier reçu en paramètre. Il sert à tester l'obtention de paramètres à l'exécution.

Assurez-vous que ces tests fonctionnent bien avant de commencer. **Pour plusieurs d'entre eux, l'ordre dans lequel les messages apparaissent ne sera pas celui des numéros devant les phrases, tout simplement parce que l'algorithme de planification du tourniquet leur est appliqué.**

Nouveaux tests :

tswap : Test qui dépasse la quantité de mémoire physique totale, avec un seul processus. Demande des échanges avec la mémoire secondaire.

tprime : Test plus exhaustif pour le remplacement de pages local avec deux processus.

tsupersqrt : Cause des échanges et des remplacements de pages avec beaucoup de processus, qui utilisent peu de mémoire chacun.

terat : Ultime test pour le devoir 4. Cause des échanges et des remplacements avec beaucoup de processus qui utilisent beaucoup de mémoire.

À propos de ThreadStats...

De nouvelles fonctionnalités ont été ajoutées à Nachos pour vous permettre de mieux voir ce qui se passe et diagnostiquer des problèmes après l'exécution.

Deux fonctions de la classe AddrSpace sont particulièrement utiles : PrintPageTable et PrintPage. La première vous affiche l'état de la table de pages, qui devrait changer régulièrement dès que vous commencez à avancer dans le travail. Vous pourrez débusquer des erreurs plus facilement en plaçant stratégiquement des appels à cette fonction. La seconde vous permettra de vérifier le contenu d'une page telle qu'elle a été chargée en mémoire physique et plus tard, avec vos modifications, de la comparer avec le contenu du fichier d'échanges. Ceci se révélera utile si vous aurez des doutes concernant votre implantation du système de mémoire virtuelle avec échanges. Ensuite, une nouvelle classe, ThreadStats, vous permet de vérifier les résultats d'exécution des processus. Dorénavant, lorsque Nachos se termine, il affiche plusieurs statistiques sur les divers processus. Entre autres, vous pouvez voir le code de sortie, souvent utilisé pour vérifier le résultat final dans les tests plus avancés. De plus, vous pouvez utiliser ThreadStats pour compter les fautes de pages et les échanges avec la mémoire secondaire. Ceci est très pratique pour voir rapidement si un test a déclenché le comportement attendu. La définition de cette classe se trouve dans Thread.h, vous devriez y jeter un coup d'œil afin de savoir comment l'utiliser lorsque vous en aurez besoin.

Tâches à réaliser

Le travail se divise en deux parties importantes.

- 1) implantation de la demande de pages
- 2) implantation du remplacement de pages

La première partie doit être réalisée par toutes les équipes, tandis que la seconde comporte certaines étapes ne concernant que les équipes de 2 et de 3.

La première partie est testable à l'aide des tests du devoir 3. À la fin de cette partie, tous les anciens tests (tcreate, terror, texec, textit, tfull, tjoin, tlog, tminijoin, tread, treadwrite, trobin, tsqrt, twrite, tyield) devraient fonctionner parfaitement.

La seconde partie est testable à l'aide des nouveaux tests (tswap, tsupersqrt, tprime, terat).

Vous pouvez taper ces commandes directement dans le répertoire /userprog. Si vous voulez savoir quels programmes sont chargés par un test en particulier, vous pouvez afficher le script avec la commande « cat » (cat tjoin par exemple, pour voir la commande faite par le test « tjoin »). Les fichiers de code source des tests sont tous dans le répertoire /tp4test et contiennent généralement des commentaires qui expliquent ce que les tests devraient faire. Vous pouvez compiler les tests directement à partir du répertoire courant à l'aide de la commande « mtest ».

Partie 1 – La demande de pages

N.B. : cette partie doit être réalisée au complet par toutes les équipes

Les modifications pour cette partie sont presque entièrement réalisées dans les fichiers addrspace.cc et exception.cc.

a) Page Fault Systématique

Script de test : tout et rien

Réalisation : 5 minutes

Faites un tour dans **AddrSpace** et modifiez l'initialisation de la table de pages de telle sorte qu'aucun cadre n'est alloué. Initialisez toutes les entrées de la table de pages avec le bit de validité à FALSE. De plus, vous ne pouvez plus charger le programme en mémoire physique durant l'initialisation de l'AddrSpace! Comme le chargement se fait avec un SysCallRead, procéder ainsi avant d'avoir alloué des cadres causerait une faute de pages avant même que l'objet AddrSpace soit complètement initialisé.

Utilisez n'importe quel script de test pour vérifier que tout programme se charge et cause immédiatement une faute de pages, pour ensuite terminer.

b) Traitement des fautes de pages : loadFromExecutable

Script de test : texit,[tlog : erreurs dans les affichages]

Réalisation : 30 minutes à 1h

Maintenant que les processus causent des fautes de pages, il faut corriger la situation en chargeant la page absente dans un cadre et en modifiant la table de pages en conséquence.

Modifiez le gestionnaire de l'erreur PageFault (exception.cc) pour faire le chargement de la page fautive au lieu de terminer le processus.

Les pages peuvent être chargées à partir du fichier exécutable. Pour ce faire, vous devez être capable de charger seulement une page de celui-ci. Vous pouvez assumer que les sections code et initData sont contigües dans le fichier et dans l'espace d'adresses (pas de vide entre les deux). La pile ne se trouve pas dans l'exécutable, mais si vous tentez de lire après la fin de fichier, vous obtiendrez des 0 (ce qui est précisément ce que vous voulez).

Écrivez donc une fonction loadFromExecutable(int) qui prend un numéro de page et charge les données correspondantes dans un cadre que vous aurez assigné à la page en question. Présentement, toutes fautes de pages déclencheront ce type de chargement; plus tard, certaines pages devront être chargées à partir du fichier d'échanges. Une fois cette étape correctement réalisée, texit devrait fonctionner. Les autres tests causeront divers problèmes. tlog devrait fonctionner en apparence, mais l'affichage va contenir une erreur. Il est probable que tsqrt fonctionne également.

N.B. Le compteur ordinal de la machine MIPS ne devrait PAS avancer après une faute de pages, résolue ou non. Si elle est résolue, alors l'instruction qui a causé la faute doit être refaite (et ne causera plus la faute de pages en question).

c) Corrigez CopyFromUser et CopyToUser

Script de test : tous les anciens tests

Réalisation : 15 minutes à 1h

Le problème qui survient dans le test précédent est dû au fait qu'une faute de pages peut survenir lors de l'accès aux données d'un processus par le noyau. En effet, comme CopyFromUser et CopyToUser font appel à ReadMem et WriteMem, qui ont besoin, évidemment, que les données soient chargées en mémoire, il est possible que la page contenant les données désirées ne soit pas encore chargée.

Si une faute de pages survient lors d'un appel à une de ces fonctions par le noyau, alors il faudra reprendre la lecture ou l'écriture demandée. Bien que votre gestion des fautes de pages ait chargé la page en question, le ReadMem ou le WriteMem a quand-même échoué.

Modifiez les fonctions du fichier copy.cc pour tenir compte de cette situation particulière. Référez-vous à la définition des fonctions ReadMem et WriteMem pour savoir comment traiter ce problème.

Après cette modification, tous les anciens tests pour le tp3 devraient fonctionner comme avant.

Partie 2 – Remplacement de pages

Les modifications pour cette partie sont presque toutes dans addrspace.cc/.h.

a) Création des fichiers d'échanges

Script de test : tous les anciens tests

Réalisation : 15 minutes à 1h.

La première étape à réaliser pour implanter un système de mémoire virtuelle est de réserver un espace pour faire des échanges avec la mémoire secondaire (fichiers sur disque).

La méthode la plus simple pour réaliser cela est de créer un fichier d'échanges par processus. Chaque processus se voit donc associer un fichier qui contiendra des copies du contenu des pages telles qu'elles étaient en mémoire lorsqu'on a décidé de les sauvegarder pour assigner leur cadre à une autre page.

L'organisation de ce fichier devrait être très simple. En fait, c'est une représentation parfaite de l'espace d'adresses tel que le processus le conçoit : complètement contigu, les pages en ordre, chaque page se trouve dans le fichier à la position (Numéro de page) * (Taille de page).

Avant de procéder à faire des échanges, vous devez créer un fichier pour chaque processus. Pour ce faire, vous devez arriver à générer de façon fiable des noms uniques pour ces fichiers, les

nettoyer correctement entre les exécutions, puis arriver à reconnaître facilement quel fichier appartenait à quel processus pour fins de débogage.

Écrivez une fonction qui génère un nom de fichier unique à partir des informations sur le processus en cours de chargement. Ce fichier devrait être créé lors de la création de l'AddrSpace, puis demeurer ouvert et valide tant que le processus n'a pas terminé. Idéalement, il faudrait effacer ce fichier lorsque le processus est détruit.

Cependant, vous pouvez remettre cette dernière étape à plus tard. Assurez-vous que vous effacez toute trace des fichiers d'échanges et des données qui y traînaient avant de faire une nouvelle exécution de nachos!!

Pour faire le ménage, vous pouvez exécuter la commande « wipe » dans le répertoire

/vm, si vos fichiers d'échange ont un nom qui termine par l'extension « .swp ».

Finalement allez voir le manuel d'instructions de la fonction « sprintf », ça sauve des vies.

b) Remplacement de pages local

Script de test : tswap

Réalisation : 2h à 16385h.

N.B. Cette partie peut vous causer quelques maux de tête si vous faites des erreurs lors de vos lectures et écritures dans le fichier d'échanges. Pour vous permettre de diagnostiquer de tels problèmes, la fonction « PrintPage » devrait être modifiée afin d'afficher le contenu du fichier d'échanges correspondant à la page demandée, pour vous permettre de voir si vous avez correctement réalisé vos échanges. De plus, la fonction « PrintPageTable » devrait vous renseigner sur l'état de la table de pages alors que vous allez y faire beaucoup de modifications durant l'exécution des processus.

Maintenant que vous avez un fichier d'échanges par processus, vous devez déterminer s'il est nécessaire de procéder à un échange avec la mémoire secondaire.

Ceci devrait se produire rapidement avec tswap. Premièrement, retirez le ASSERT qui interdit le chargement de programmes plus gros que la mémoire physique dans le constructeur d'AddrSpace... et regardez votre nachos se planter dans le mur quand le bitmap qui conserve la liste des cadre libre va vous sortir -1 lors de sa recherche de cadre libre...

Vous devez donc traiter le cas où il n'y a plus de cadres libres lors d'une faute de pages.

Une façon simple de régler ce problème est de trouver une entrée dans la table de pages du processus courant qui est valide, faire une copie de sûreté du contenu de la page dans le fichier d'échanges (un swapOut), puis rendre la page invalide et assigner le cadre libéré à la page ayant causé une faute. Finalement, la page fautive peut être chargée dans le cadre qu'on vient de lui assigner (swapIn).

Comme la page qui est victime d'un swapOut est une des pages du processus ayant causé la faute de pages, on dit que le remplacement est local (les échanges se font entre les pages d'un même processus). Un processus ne peut pas voler un cadre à un autre processus.

Pour choisir la page victime, un algorithme simpliste peut fonctionner : la première page valide trouvée fera l'affaire. Le problème, c'est qu'il ne faudrait pas toujours commencer à boucler sur la table de pages à la page 0, parce que certaines pages seront trop souvent des victimes, ce qui va faire exploser la quantité de fautes de pages. Il serait mieux de conserver un indicateur d'emplacement où la dernière victime a été choisie, l'avancer et retourner au début de la table lorsqu'on l'a parcourue au complet. Parmi les algorithmes vus en classe, c'est semblable à l'algorithme FIFO. Ensuite, il convient d'implanter deux fonctions : swapIn et swapOut, permettant respectivement de récupérer le contenu d'une page du fichier d'échanges et de sauvegarder le contenu d'une page dans le fichier d'échanges.

Finalement, il faut pouvoir faire la distinction entre une page dont la version la plus à jour est dans le fichier d'échanges, et une dont la version la plus à jour est dans l'exécutable. Lorsqu'une page est invalide, vous pouvez utiliser une valeur spéciale dans le no de cadre (physicalPage) pour indiquer cela. Au départ, une page qui n'a jamais été chargée est dans l'exécutable, alors que si elle a été échangée au moins une fois, la copie la plus à jour est dans le fichier d'échanges.

Vous pouvez toutefois tenter de simplifier cette partie du procédé en copiant tout le contenu de l'exécutable (excepté l'en-tête) dans le fichier d'échanges lors de la construction de l'AddrSpace et considérer que les pages sont dans le fichier d'échanges depuis le début.

c) Remplacement de pages à plusieurs processus

*Script de test : tprime et tsupersqrt**

Réalisation : 0 minutes à ?h.

Cette partie doit être réalisée par les équipes de 2 et 3.

Cette étape devrait normalement être déjà réalisée, cependant, il est probable que des bugs dans l'implantation de la partie b) ne se révèlent que lors de l'exécution de **tprime**. En effet, le test précédent ne réutilise pas la majorité des pages, alors qu'ici, on va s'en servir plusieurs centaines de fois. Vos échanges doivent fonctionner parfaitement pour que ce test passe.

Pour que la réalisation de cette partie soit acceptable, tprime doit faire son travail correctement.

Ce programme calcule le nombre de nombres premiers précédant 100001. Le résultat devrait être exactement **9592** (code de sortie des deux processus). Il utilise un crible d'Ératostène pour faire le travail, donc un très grand tableau statique, qui n'entre pas dans la mémoire physique. De plus, le programme cohabite avec son processus parent, qui tente de récupérer le résultat. Ce programme devrait causer des milliers de fautes de pages et d'échanges.

Le test `tsupersqrt` peut fonctionner lors de cette étape, mais ce n'est pas complètement certain, ça peut dépendre de plusieurs facteurs.

d) Nouveaux processus et mémoire pleine

Script de test : `tsupersqrt`, `terat`

Réalisation : 1h à 24h.

Cette partie doit être réalisée par les équipes de 2 et 3.

Il ne manque maintenant plus qu'une étape avant d'avoir une mémoire virtuelle complètement utilisable, régler le cas des processus qui ne possèdent aucun cadre. En effet, lorsqu'on exécute beaucoup de processus et que certains commencent un peu tardivement, il est possible qu'ils causent une faute de pages alors qu'aucun cadre n'est disponible dans la mémoire ET qu'ils n'ont jamais eu aucun cadre assigné!

Ceci implique que le remplacement local est insuffisant : on ne peut pas trouver de victimes parmi les pages valides du processus courant, il devient nécessaire d'en voler une à un autre processus.

Deux approches sont possibles pour corriger le problème :

- 1) Chercher parmi les processus un qui a une page valide et faire le remplacement avec cette page;
- 2) Implanter le remplacement global.

Si vous êtes une équipe de 3 ou que vous désirez faire le bonus, vous devriez directement passer au no. e) et utiliser la seconde approche.

Si vous désirez seulement coder une patch pour ce cas particulier, alors tentez la première approche. Attention, cependant, ça peut devenir assez mêlant de retrouver une page à échanger, réfléchissez bien en codant votre algorithme. Si seulement vous aviez la liste des cadres, en ordre, avec quel processus les possède...

Le résultat du test `tsupersqrt` devrait être **67**. Le résultat de `terat` devrait être **13849**.

e) Remplacement global

Script de test : `terat`

Réalisation : `have fun`.

Cette partie est un bonus pour les équipes de 1 et 2 (10% de la note), mais obligatoire pour les équipes de 3.

Finalement, un test pour mettre votre implantation à l'épreuve.

Si vous n'avez appliqué qu'une patch pour le no d), alors l'exécution de **terat** devrait être plutôt pénible.

Ce test est semblable à tprime, mais en 10x plus lourd. Il divise le tableau de nombres premiers à cribler en 15 morceaux de 10 000 entrées, puis démarre un processus qui va cribler chaque partie. Chacun des processus doit d'abord générer un mini-tableau de nombres premiers inférieurs à la racine du nombre maximal à cribler, et pour ça, il faut calculer la racine carrée du nombre maximal, et celle du plus grand nombre premier potentiel qui sera inférieur à cette racine. En gros: beaucoup de parcours de tableaux, beaucoup de création de sous-processus, des fonctions d'affichage lourdes, c'est la fête!!!

S'il y a un bug dans votre implantation, ce test a beaucoup de chances de le révéler. Pour vous assurer d'avoir une bonne implantation pour résoudre ce test, vous devez implanter une structure de données qui remplacera la variable freeFrame. Au lieu d'un bitmap, vous aurez alors une association directe entre chaque no de cadre, un processus et la page de ce processus qui se trouve dans le cadre. Votre choix de victime se fera maintenant globalement, en parcourant cette liste de cadres. Le remplacement devrait pouvoir se faire en temps $O(1)$. Comme chaque processus risque de causer environ 50 000 à 70 000 fautes de pages, la recherche d'un cadre à voler ne doit pas être trop lente...